Segmented Semantic Program Evolution

A compact technical note for the OpenROAD timing/placement evolution framework

Goal. Evolve real C++ decision logic inside OpenROAD while keeping the mutation surface bounded, auditable, and testable through the full OpenROAD-flow-scripts flow.

Core idea. The framework does not randomly rewrite the whole tool. It generates a small C++ policy that changes algorithmic scoring/order decisions, then compiles and evaluates that candidate in OpenROAD.

Real source evolution, not only parameter tuning

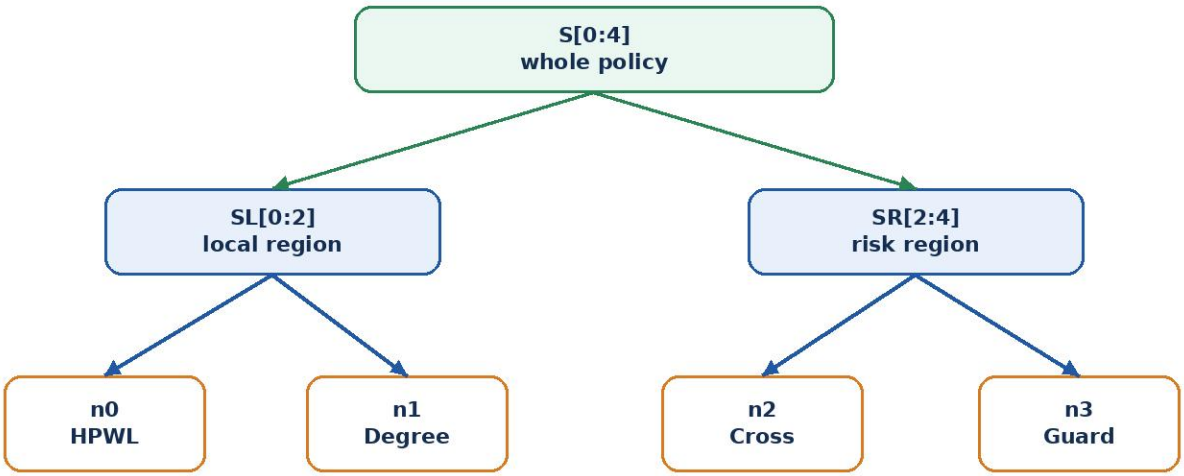
Candidate = generated C++ policy + deterministic ID + archived metrics

- Evolved code is inserted into a constrained OpenROAD decision point such as OptMirror candidate ranking.
- Legality and final physical checks remain in OpenROAD and ORFS.
- Every candidate is compiled, tested, run through the flow, scored, and stored permanently.

1. Program Representation

The evolved code is represented as blocks, a segment tree, and a semantic relation graph.

Each generated C++ statement block is treated as a semantic leaf. Examples include HPWL pressure, pin-degree pressure, fanout shock guard, entropy tempering, and deterministic tie shaping.



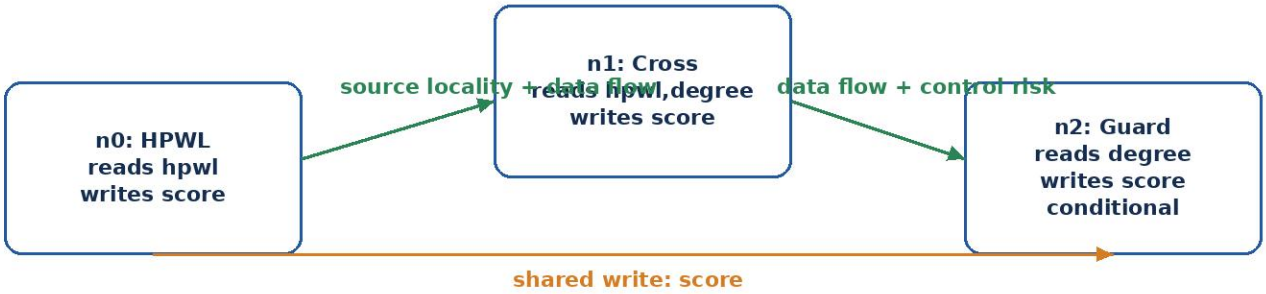
The segment tree gives hierarchical edit regions. Mutations can replace a leaf, insert a block, delete a block, swap two related blocks, reverse a subtree, or duplicate a subtree.

Why this matters
The search operates on structured program regions instead of raw text, making the evolved source easier to bound, explain, replay, and review.

2. Semantic Relation Graph

Edges capture why two code blocks should be evolved together.

For every code block, the framework records reads, writes, control role, and heuristic role. It then adds graph edges between blocks that are semantically related.



- Source-locality: neighboring blocks in the generated C++ program.
- Data-flow: blocks share variables such as hpwl, degree, stable_id, entropy, or score.
- Control role: blocks have similar control structure, such as straight-line scoring or conditional guards.

Graph-aware mutation
The search can swap related blocks or insert bridge blocks along these edges, so evolution is structured and deterministic rather than raw text editing.

3. Evolution Loop

Candidate generation is deterministic, resumable, and archive-driven.



Selection. Valid candidates are ranked by score. The best few archived candidates become parents for later mutations. Invalid candidates are kept in the archive with failure reasons but are not selected as parents.

Memory. The archive file is append-only. Running N=40 today and N=80 later gives 120 total recorded attempts; earlier results are reused instead of overwritten.

Main operators
replace leaf - insert block - delete block - swap related blocks - reverse segment - duplicate segment - insert semantic bridge

4. Correctness and Evaluation

The framework uses constrained evolution plus empirical end-to-end validation.

Correctness model. The framework does not attempt full C++ equivalence, which is intractable for arbitrary programs. Instead, it restricts what can change and then validates the result through compilation, regression, and full physical-design flow execution.

- Restricted surface: evolved code changes only a generated scoring/ranking policy.

Evaluation metrics
Setup WNS/TNS - Hold WNS/TNS - Route wirelength - runtime - physical violation indicators

A candidate is promotable only if it is valid and improves the weighted objective against the stock baseline.

- Final metrics must be physically valid before scoring.

Outputs. archive.jsonl stores every attempt. summary.md reports original code vs evolved code, candidate IDs, generated-policy summary, metrics, deltas, and promotion status.